📋 Relatório Final — Protótipo de Compilador TinyScript

> **Disciplina:** Teoria da Computação e Compiladores — UNIBH 2026/1
> **Professora:** Charlene Cássia de Resende
> **Aluno:** Sérgio Pinton Pavanelli | **RA:** 123220202
> **Entregável 4:** Relatório final contendo análise, geração de código e tecnologias utilizadas
> **Data:** Abril de 2026

📑 Sumário

1. Introdução
2. Tecnologias Utilizadas
3. Fase 1 — Análise Léxica
4. Fase 2 — Análise Sintática
5. Fase 3 — Geração de Código
6. Integração e Interface CLI
7. Testes e Resultados
8. Análise Crítica
9. Conclusão
10. Referências Bibliográficas

1. Introdução

1.1 Contexto e Motivação

A crescente complexidade das linguagens de programação modernas exige dos engenheiros de software uma compreensão sólida dos fundamentos de compiladores. Ferramentas como GCC, LLVM/Clang e CPython são amplamente utilizadas, porém raramente compreendidas em sua estrutura interna.

Este projeto tem como objetivo construir um **protótipo funcional de compilador** para uma linguagem imperativa minimalista denominada **TinyScript**, cobrindo as três fases fundamentais de compilação:

1. **Análise Léxica** — reconhecimento de tokens
2. **Análise Sintática** — verificação estrutural e construção da AST
3. **Geração de Código** — produção de código intermediário de três endereços (TAC)

1.2 Objetivos

- ✅ Implementar um lexer completo para TinyScript
- ✅ Implementar um parser recursivo descendente com construção de AST
- ✅ Implementar um gerador de código intermediário TAC
- ✅ Produzir estudo de caso sobre compilador real (CPython)
- ✅ Documentar a arquitetura e as decisões de projeto

1.3 Delimitação do Escopo

Item	Incluso	Justificativa
Análise léxica	✅ Sim	Fase 1 obrigatória
Análise sintática	✅ Sim	Fase 2 obrigatória
Geração de código TAC	✅ Sim	Fase 3 obrigatória
Análise semântica completa	❌ Não	Fora do escopo da A3
Geração de código nativo	❌ Não	Requer backend (LLVM/GCC)
Otimizações	❌ Não	Fase avançada

2. Tecnologias Utilizadas

2.1 Linguagem de Implementação

Tecnologia	Versão	Papel no projeto
Python	3.10+	Linguagem de implementação do compilador
Git	2.x	Controle de versão do código-fonte
VS Code	1.x	Ambiente de desenvolvimento

Justificativa da escolha do Python:

- Sintaxe clara e próxima do pseudocódigo — ideal para fins didáticos
- Suporte nativo a `enum`, `dataclass`, programação orientada a objetos
- Fácil introspecção (módulos `ast`, `dis`, `tokenize`) para comparação com CPython
- Sem necessidade de compilação — ciclo rápido de desenvolvimento

2.2 Paradigmas e Padrões Utilizados

Padrão	Aplicação
Visitor Pattern	`GeradorDeCodigo` — visitação da AST por tipo de nó
Recursive Descent Parsing	`Parser` — cada não-terminal é um método Python

Padrão	Aplicação
State Machine	`Lexer` — transições de estado para reconhecimento de lexemas
Composite Pattern	`ASTNode` — árvore hierárquica de nós

2.3 Módulos Python (stdlib) Utilizados

Módulo	Uso
`enum`	Definição de `TokenType`
`sys` , `os`	Interface CLI e manipulação de caminhos

> **Nenhuma biblioteca externa** foi utilizada — o compilador é 100% Python puro.

3. Fase 1 — Análise Léxica

3.1 Conceito

A análise léxica (também chamada de **tokenização** ou **scanning**) é a primeira fase do compilador. Ela agrupa caracteres do código-fonte em unidades significativas chamadas **tokens**, descartando espaços e comentários.

Formalmente, um token é um par `(tipo, valor)` . O processo de reconhecimento de tokens é governado por **expressões regulares** ou **autômatos finitos determinísticos (AFD)**.

3.2 Implementação

Arquivo: `src/lexer.py`

O `Lexer` mantém um ponteiro de posição e avança caractere a caractere, implementando manualmente o AFD para cada categoria de token:

```
Estado inicial
    ├── dígito (0-9)  →  loop: acumula dígitos  →  Token(INTEGER)
    ├── letra/_ →  loop: acumula alfanum  →  verifica palavras reservadas  →
Token(IDENT|IF|ELSE|WHILE|PRINT)
    ├── '+'  →  Token(PLUS)
    ├── '-'  →  Token(MINUS)
    ├── '*'  →  Token(STAR)
    ├── '/'  →  lookahead: próximo '/'?  →  comentário (ignorar)  →  Token(SLASH)
    ├── '='  →  lookahead: próximo '='?  →  Token(EQ)  :  Token(ASSIGN)
    ├── '!'  →  lookahead: próximo '='?  →  Token(NEQ)  :  ErroLexico
    ├── '<'  →  lookahead: próximo '='?  →  Token(LE)   :  Token(LT)
    ├── '>'  →  lookahead: próximo '='?  →  Token(GE)   :  Token(GT)
    ├── '('  →  Token(LPAREN)
```

```
    ├── ')'  →  Token(RPAREN)
    ├── '{'  →  Token(LBRACE)
    ├── '}'  →  Token(RBRACE)
    ├── ';'  →  Token(SEMI)
    ├── espaço/\t/\n/\r  →  ignorar
    └── outros  →  ErroLexico(char, linha, coluna)
```

3.3 Rastreamento de Posição

Cada token armazena linha e coluna de origem, possibilitando mensagens de erro precisas:

```
Token(tipo=TokenType.IDENT, valor='x', linha=3, coluna=5)
```

3.4 Exemplo de Saída

Entrada: `soma = a + 10;`

```
Token(IDENT,   'soma', L1:C1)
Token(ASSIGN,  '=',    L1:C6)
Token(IDENT,   'a',    L1:C8)
Token(PLUS,    '+',    L1:C10)
Token(INTEGER, 10,     L1:C12)
Token(SEMI,    ';',    L1:C14)
Token(EOF,     None,   L1:C15)
```

4. Fase 2 — Análise Sintática

4.1 Conceito

A análise sintática verifica se a sequência de tokens respeita a gramática da linguagem e constrói a **Árvore Sintática Abstrata (AST)** — uma representação hierárquica do programa que elimina detalhes desnecessários (parênteses redundantes, ponto e vírgula, etc.).

A gramática da TinyScript pertence à classe **LL(1)** — pode ser analisada de cima para baixo com apenas 1 token de lookahead.

4.2 Estratégia: Parser Recursivo Descendente

Arquivo: `src/parser.py`

Cada não-terminal da gramática corresponde a um método Python. A hierarquia de chamadas reflete diretamente a gramática:

```
parse()
└── _parse_program()
    └── _parse_statement() [repetido]
        ├── _parse_assignment()
        │   └── _parse_expression()
        ├── _parse_if()
        │   ├── _parse_expression()
        │   ├── _parse_block()
        │   └── _parse_block()  (else, opcional)
        ├── _parse_while()
        │   ├── _parse_expression()
        │   └── _parse_block()
        └── _parse_print()
            └── _parse_expression()

_parse_expression()
└── _parse_comparison()
    └── _parse_additive()
        └── _parse_multiplicative()
            └── _parse_unary()
                └── _parse_primary()
```

4.3 Lookahead e Decisão de Regra

O parser usa 1 token de lookahead para decidir qual regra aplicar:

Token atual	Próximo token	Regra escolhida
`IF`	—	`_parse_if()`
`WHILE`	—	`_parse_while()`
`PRINT`	—	`_parse_print()`
`IDENT`	`ASSIGN`	`_parse_assignment()`

4.4 Nós da AST

Arquivo: `src/ast_nodes.py`

```
ASTNode
├── Program(statements: list)
├── Block(statements: list)
├── Assignment(name: str, value: ASTNode)
├── IfStatement(condition, then_block, else_block?)
├── WhileLoop(condition, body)
├── PrintStatement(expression)
├── BinaryOp(left, op: str, right)
├── UnaryOp(op: str, operand)
```

```
├── IntegerLiteral(value: int)
└── Variable(name: str)
```

4.5 Exemplo de AST

Entrada:

```
fat = 1;
while (n > 1) {
    fat = fat * n;
    n = n - 1;
}
```

AST gerada:

```
Program
├── Assignment [fat]
│   └── IntegerLiteral [1]
└── WhileLoop
    ├── condição: BinaryOp [>]
    │   ├── Variable [n]
    │   └── IntegerLiteral [1]
    └── corpo: Block
        ├── Assignment [fat]
        │   └── BinaryOp [*]
        │       ├── Variable [fat]
        │       └── Variable [n]
        └── Assignment [n]
            └── BinaryOp [-]
                ├── Variable [n]
                └── IntegerLiteral [1]
```

5. Fase 3 — Geração de Código

5.1 Conceito

A geração de código transforma a AST em uma representação mais próxima da linguagem-alvo. Neste projeto, utilizamos o **Código de Três Endereços (Three-Address Code — TAC)**, formato intermediário amplamente estudado na literatura (AHO et al., cap. 6).

Uma instrução TAC tem a forma geral:

```
resultado = operando1  operador  operando2
```

5.2 Implementação: Padrão Visitor

Arquivo: `src/codegen.py`

O `GeradorDeCodigo` usa o padrão **Visitor** via despacho dinâmico:

```python
def _visitar(self, no):
    nome_metodo = '_visitar_' + type(no).__name__
    metodo = getattr(self, nome_metodo)
    return metodo(no)
```

Isso evita longas cadeias de `isinstance` e facilita a extensão (basta adicionar `_visitar_NovoTipo`).

5.3 Variáveis Temporárias e Rótulos

Elemento	Formato	Exemplo	Finalidade
Temporário	`_t{n}`	`_t0`, `_t5`	Armazenar resultado intermediário de expressão
Rótulo	`L{n}`	`L0`, `L3`	Marcar destino de desvios condicionais/incondicionais

5.4 Padrões de Geração

Atribuição simples:

```
// TinyScript: x = 42;
_t0 = 42
x = _t0
```

Operação binária:

```
// TinyScript: soma = a + b;
_t0 = a + b
soma = _t0
```

Condicional if/else:

```
// TinyScript: if (x > 0) { print(x); } else { print(0); }
_t0 = 0
_t1 = x > _t0
IF NOT _t1 GOTO L0
PRINT x
GOTO L1
L0:
    _t2 = 0
```

```
    PRINT _t2
L1:
```

Laço while:

```
// TinyScript: while (i < 10) { i = i + 1; }
L0:
    _t0 = 10
    _t1 = i < _t0
    IF NOT _t1 GOTO L1
    _t2 = 1
    _t3 = i + _t2
    i = _t3
    GOTO L0
L1:
```

6. Integração e Interface CLI

6.1 Orquestrador

Arquivo: `src/compiler.py`

A classe `CompiladorTinyScript` integra as três fases e expõe os resultados:

```python
compilador = CompiladorTinyScript()
resultado  = compilador.compilar_arquivo("programa.tiny")

resultado['tokens']  # List[Token]
resultado['ast']     # Program (AST)
resultado['codigo']  # str (código intermediário TAC)
```

6.2 Uso via Linha de Comando

```
python src/compiler.py examples/exemplo1_aritmetica.tiny
```

Saída produzida:

```
  Compilando: examples/exemplo1_aritmetica.tiny
────────────────────────────────────────────────────────────


═══════════════════════════════════════════════════════════
  FASE 1 — ANÁLISE LÉXICA  |  Tokens produzidos
═══════════════════════════════════════════════════════════
```

```
  [IDENT     ]  valor='a'            L1:C1
  [ASSIGN    ]  valor='='            L1:C3
  [INTEGER   ]  valor=10             L1:C5
  ...

═══════════════════════════════════════════════════════════
  FASE 2 — ANÁLISE SINTÁTICA  |  AST gerada
═══════════════════════════════════════════════════════════
  Program
    Assignment  [a]
      IntegerLiteral  [10]
  ...

═══════════════════════════════════════════════════════════
  FASE 3 — GERAÇÃO DE CÓDIGO  |  Código intermediário (TAC)
═══════════════════════════════════════════════════════════
      _t0 = 10
      a = _t0
  ...
```

7. Testes e Resultados

7.1 Exemplo 1 — Aritmética (`exemplo1_aritmetica.tiny`)

Entrada:

```
a = 10;
b = 3;
soma = a + b;
print(soma);
```

Código TAC gerado:

```
_t0 = 10
a = _t0
_t1 = 3
b = _t1
_t2 = a + b
soma = _t2
PRINT soma
```

Análise: As 4 instruções de atribuição geram temporários corretamente. A instrução `print` referencia `soma` diretamente (sem temporário extra), pois `Variable` retorna o nome sem emitir instrução — comportamento eficiente e correto.

7.2 Exemplo 2 — Condicional (`exemplo2_condicional.tiny`)

Entrada:

```
nota = 75;
media = 60;
if (nota >= media) {
    resultado = 1;
    print(resultado);
} else {
    resultado = 0;
    print(resultado);
}
```

Código TAC gerado (trecho):

```
_t0 = 75
nota = _t0
_t1 = 60
media = _t1
_t2 = nota >= media
IF NOT _t2 GOTO L0
_t3 = 1
resultado = _t3
PRINT resultado
GOTO L1
L0:
_t4 = 0
resultado = _t4
PRINT resultado
L1:
```

Análise: A estrutura if/else gera corretamente dois rótulos (`L0` = else, `L1` = fim). O desvio `IF NOT ... GOTO L0` implementa a semântica de "pular o bloco then se a condição for falsa".

7.3 Exemplo 3 — Laço While (`exemplo3_laco.tiny`)

Entrada (fatorial):

```
n = 5;
fat = 1;
while (n > 1) {
    fat = fat * n;
    n = n - 1;
}
print(fat);
```

Código TAC gerado:

```
_t0 = 5
n = _t0
_t1 = 1
fat = _t1
L0:
_t2 = 1
_t3 = n > _t2
IF NOT _t3 GOTO L1
_t4 = fat * n
fat = _t4
_t5 = 1
_t6 = n - _t5
n = _t6
GOTO L0
L1:
PRINT fat
```

Análise: O laço while gera o padrão clássico com rótulo de início (`L0:`), desvio de saída (`IF NOT ... GOTO L1`) e retorno ao início (`GOTO L0`). Para `n=5` , o resultado correto é `fat=120` .

7.4 Teste de Erros Léxicos e Sintáticos

Entrada	Fase	Mensagem de erro esperada
`@x = 1;`	Léxica	`Erro léxico na linha 1, coluna 1: caractere inesperado '@'`
`x = ;`	Sintática	`Erro sintático: expressão inválida: token inesperado SEMI`
`if x > 1 { }`	Sintática	`Erro sintático: esperado LPAREN, encontrado IDENT`
`while (x > 1) { x = 2`	Sintática	`Erro sintático: bloco não fechado — esperado '}'`

8. Análise Crítica

8.1 Comparação com Ferramentas Modernas

Aspecto	TinyScript	CPython	GCC/LLVM
Análise Léxica	AFD manual	AFD automático (re)	Flex / lex automático
Análise Sintática	Recursivo descendente manual	PEG Parser gerado	Recursivo descendente
Representação intermediária	TAC (texto)	Bytecode (binário)	LLVM IR (SSA)

Aspecto	TinyScript	CPython	GCC/LLVM
Otimizações	Nenhuma	Peephole	Múltiplos passes (O1-O3)
Verificação semântica	Não implementada	Sim (symtable)	Sim (tipos, escopo)
Geração de código nativo	Não	VM (bytecode)	Sim (x86/ARM/etc)

8.2 Decisões de Projeto

1. Python como linguagem de implementação
Escolhido pela legibilidade e facilidade de prototipagem. Em um compilador de produção, o front-end seria em C ou Rust para maior desempenho.

2. TAC em vez de bytecode
O TAC textual é mais legível para fins didáticos do que bytecode binário. É também o formato usado como entrada para back-ends como LLVM IR (após conversão para SSA).

3. Parser recursivo descendente
Mais simples de implementar e depurar que parsers LR/LALR. Adequado para gramáticas LL(k) como TinyScript. Ferramentas como `yacc` / `bison` geram parsers LALR automaticamente para gramáticas mais complexas.

4. Sem tabela de símbolos / verificação semântica
A análise semântica (verificação de variáveis não declaradas, tipos incompatíveis) foi deixada fora do escopo desta versão para manter o foco nas fases léxica, sintática e de geração de código.

8.3 Limitações Identificadas

- TinyScript suporta apenas inteiros (sem `float`, `string`, `boolean`)
- Sem suporte a funções/procedimentos
- Sem análise semântica (uso de variável não declarada não gera erro)
- O gerador de código não realiza otimizações (ex.: eliminação de temporários redundantes `_t0 = 42; x = _t0 → x = 42`)

9. Conclusão

Este projeto demonstrou na prática os fundamentos da construção de compiladores ao implementar um protótipo funcional para a linguagem TinyScript. As três fases clássicas de compilação foram implementadas com sucesso:

Fase	Módulo	Status
Análise Léxica	`lexer.py`	✅ Funcional

Fase	Módulo	Status
Análise Sintática	`parser.py + ast_nodes.py`	✅ Funcional
Geração de Código TAC	`codegen.py`	✅ Funcional
Interface CLI	`compiler.py`	✅ Funcional

O estudo de caso sobre CPython revelou como um compilador de produção expande cada uma dessas fases com otimizações, análise semântica e geração de código nativo/bytecode. A comparação com GCC e LLVM evidenciou a riqueza do ecossistema moderno de compiladores e os trade-offs entre portabilidade, desempenho e complexidade de implementação.

Lições aprendidas:

- A separação clara de responsabilidades entre fases facilita manutenção e extensão
- O padrão Visitor torna o gerador de código extensível sem modificar os nós da AST
- Compiladores reais combinam centenas de passes de otimização sobre representações intermediárias sofisticadas (SSA, CFG)
- Ferramentas como LLVM democratizaram a criação de compiladores de produção ao prover backends reutilizáveis

10. Referências Bibliográficas

1. **AHO, Alfred V.; LAN, Monica S.; SETHI, Ravi; ULLMAN, Jeffrey D.** *Compiladores: princípios, técnicas e ferramentas.* 2. ed. São Paulo: Pearson Addison-Wesley, 2008. *(Dragon Book)*

2. **MENDEZ, P.; BLAUTH, P.** *Linguagens formais e autômatos.* 6. ed. Porto Alegre: Bookman, 2010.

3. **SANTOS, Pedro Reis; LANGLOIS, Thibault.** *Compiladores: da teoria à prática.* Rio de Janeiro: LTC, 2019.

4. **GRUNE, Dick; JACOBS, Ceriel J. H.** *Parsing Techniques: A Practical Guide.* 2. ed. New York: Springer, 2008.

5. **CPython Developers.** *CPython Source Repository.* Disponível em: https://github.com/python/cpython. Acesso em: abr. 2026.

6. **Python Software Foundation.** *Python Developer's Guide — Compiler.* Disponível em: https://devguide.python.org/internals/compiler/. Acesso em: abr. 2026.

7. **LATTNER, Chris; ADVE, Vikram.** *LLVM: A Compilation Framework for Lifelong Program Analysis & Transformation.* In: Proceedings of CGO 2004. IEEE, 2004.